

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

6-mashg’ulot. Pythonda takrorlanuvchi jarayonlarni dasturlash.

O‘quv savollari:

1. Sikl operatorlari – For va while bilan ishlash.
2. Break, continue va else operatorlarining qo‘llanilishi.

1-o‘quv savoli: Sikl operatorlari – For va while bilan ishlash.

Nega sikl kerak?

Takrorlanuvchi ishlarni avtomatlashtirish uchun: ro‘yxat elementlarini ko‘rish, yig‘indi hisoblash, filtr qilish, fayl satrlarini ko‘rib chiqish va hokazo.

for sikli

Sintaksis

```
for element in ketma_ketlik:  
    # takrorlanuvchi ishlar
```

ketma_ketlik — ro‘yxat (list), matn (str), to‘plam (set), lug‘at (dict), range() va hokazo bo‘lishi mumkin.

range() bilan sanash

range sonlar ketma-ketligini hosil qiladi (oxirgi chegara **kirmaydi**).

```
for i in range(5):           # 0,1,2,3,4  
    print(i)  
  
for i in range(2, 7):       # 2,3,4,5,6  
    print(i)  
  
for i in range(10, 0, -2):  # 10,8,6,4,2  
    print(i)
```

Indeks va qiymatni birga olish: enumerate()

```
mevalar = ["olma", "banan", "o'rik"]  
for indeks, nom in enumerate(mevalar, start=1):  
    print(indeks, nom)  
# 1 olma  
# 2 banan  
# 3 o'rik
```

Parallel yurish: zip()

Bir nechta ro'yxatni yonma-yon aylanadi (eng qisqa uzunlikka qadar).

```
mahsulotlar = ["non", "sut", "tuxum"]
narxlar      = [6000, 9000, 14000]
for nom, narx in zip(mahsulotlar, narxlar):
    print(f"{nom}: {narx} so'm")
```

zip() nima qiladi?

- Python'da **zip()** ikki yoki undan ortiq ro'yxatlarni (yoki ketma-ketliklarni) yonma-yon "birlashtiradi".
- Har bir elementini navbatma-navbat olib, **juftlik (tuple)** shaklida qaytaradi.
- Eng qisqa ro'yxat tugagandan keyin sikl ham tugaydi.

Misol

```
mahsulotlar = ["non", "sut", "tuxum"]
narxlar      = [6000, 9000, 14000]

for nom, narx in zip(mahsulotlar, narxlar):
    print(f"{nom}: {narx} so'm")
```

Bosqichma-bosqich ishlash jarayoni:

1. `zip(mahsulotlar, narxlar)` quyidagini hosil qiladi:
2. `[("non", 6000), ("sut", 9000), ("tuxum", 14000)]`

(aslida bu ro'yxat emas, iterator — lekin siklda shunday ko'rinadi).

3. `for nom, narx in zip(...)` — bu degani:
 - o birinchi juftlikdan `nom = "non", narx = 6000` olinadi,
 - o keyingi juftlikdan `nom = "sut", narx = 9000`,
 - o va hokazo.
4. Har safar `print(f"{nom}: {narx} so'm")` bajariladi.

Natija ekranda:

```
non: 6000 so'm
sut: 9000 so'm
tuxum: 14000 so'm
```

Muhim eslatma

Agar ro'yxatlar uzunligi turlicha bo'lsa, **eng qisqa** ro'yxat tugashi bilan sikl to'xtaydi:

```
mahsulotlar = ["non", "sut", "tuxum", "choy"]
narxlar      = [6000, 9000]

for nom, narx in zip(mahsulotlar, narxlar):
    print(nom, narx)
```

```
# faqat ikkita juftlik chiqadi:  
# non 6000  
# sut 9000
```

zip() + enumerate()

Baʼzan elementlarni nafaqat yonma-yon olish, balki **raqamlab chiqish** ham kerak boʻladi. Bunda `enumerate()` yordam beradi.

```
mahsulotlar = ["non", "sut", "tuxum"]  
narxlar      = [6000, 9000, 14000]  
  
for tartib, (nom, narx) in enumerate(zip(mahsulotlar, narxlar), start=1):  
    print(f"{tartib}) {nom}: {narx} so'm")
```

Natija:

```
1) non: 6000 so'm  
2) sut: 9000 so'm  
3) tuxum: 14000 so'm
```

Bu usulda `zip()` roʻyxatlarni birlashtiradi, `enumerate()` esa har bir juftlikka tartib raqamini beradi.

zip() + dict() (lugʻat yasash)

Koʻpincha mahsulot va narxlarni yonma-yon olib, **lugʻat (dict)** koʻrinishida saqlash qulay boʻladi.

```
mahsulotlar = ["non", "sut", "tuxum"]  
narxlar      = [6000, 9000, 14000]  
  
mahsulot_narx = dict(zip(mahsulotlar, narxlar))  
print(mahsulot_narx)
```

Natija:

```
{'non': 6000, 'sut': 9000, 'tuxum': 14000}
```

Endi lugʻatdan osongina narxlarni chaqirish mumkin:

```
print(mahsulot_narx["sut"]) # 9000
```

Qoʻshimcha – zip(*...) yordamida “teskarilash”

`zip()` ning teskari ishlatilishi ham mumkin:

Masalan, juftliklar roʻyxatini yana ikki alohida roʻyxatga ajratish:

```
juftliklar = [("non", 6000), ("sut", 9000), ("tuxum", 14000)]  
mahsulotlar, narxlar = zip(*juftliklar)  
  
print(mahsulotlar) # ('non', 'sut', 'tuxum')  
print(narxlar)     # (6000, 9000, 14000)
```

Bu ko‘pincha fayl yoki bazadan kelgan juftliklarni qayta ajratishda foydali.

`zip()` funksiyasini **uchta yoki undan ortiq ro‘yxat** bilan ishlatishni ko‘rib chiqamiz.

Misol: mahsulot, narx va miqdor

```
mahsulotlar = ["non", "sut", "tuxum"]
narxlar      = [6000, 9000, 1400]
miqdorlar    = [5, 2, 30]

for nom, narx, son in zip(mahsulotlar, narxlar, miqdorlar):
    summa = narx * son
    print(f"{nom}: {son} dona, 1 dona narxi {narx} so'm, jami = {summa} so'm")
```



Natija:

```
non: 5 dona, 1 dona narxi 6000 so'm, jami = 30000 so'm
sut: 2 dona, 1 dona narxi 9000 so'm, jami = 18000 so'm
tuxum: 30 dona, 1 dona narxi 1400 so'm, jami = 42000 so'm
```

Muhim eslatma

- `zip()` **eng qisqa ro‘yxat** uzunligiga qadar ishlaydi. Masalan, agar `mahsulotlar` ro‘yxati uzunroq bo‘lsa, qolgan elementlar ko‘rilmaydi.
- Agar barcha ro‘yxatlar uzunligini tenglashtirish kerak bo‘lsa, `itertools.zip_longest()` dan foydalanish mumkin.

Qo‘shimcha: Lug‘at yasash

Ko‘p hollarda bunday ma’lumotlardan lug‘at (dict) tuzish qulay bo‘ladi.

```
mahsulot_info = list(zip(mahsulotlar, narxlar, miqdorlar))
print(mahsulot_info)
```



Natija:

```
[('non', 6000, 5), ('sut', 9000, 2), ('tuxum', 1400, 30)]
```

Lug‘atlar bilan

```
kurs = {"USD":12600, "EUR":13800}
for kalit in kurs:                # kalitlar
    print(kalit)

for qiymat in kurs.values():      # qiymatlar
    print(qiymat)

for kalit, qiymat in kurs.items(): # juftlik
    print(kalit, qiymat)
```

Matn (string) bo‘ylab

```
s = "Toshkent"
unli = 0
for belgi in s.lower():
    if belgi in "aeiouo'u":
        unli += 1
print(unli)
```

Fayl satrlari bo‘ylab

```
with open("log.txt", "r", encoding="utf-8") as f:
    for satr in f:
        print(satr.strip())
```

while sikli

Sintaksis

```
while shart:
    # shart True bo'lsa ishlaydi
```

while — takrorlar soni oldindan noma'lum bo'lganda yoki holat o'zgarishiga bog'liq vazifalarda qulay.

Oddiy misol: hisoblagich

```
i = 1
while i <= 5:
    print(i)
    i += 1 # i ni yangilashni unutmang!
```

“Sentinel” (to‘xtatuvchi qiymat) bilan

```
jami = 0
kiritma = input("Narx kiriting (yoki 'stop'): ")
while kiritma != "stop":
    jami += int(kiritma)
    kiritma = input("Narx kiriting (yoki 'stop'): ")
print("Jami:", jami)
```

Qachon qaysi biri?

- **for**: elementlar to‘plami ma’lum bo‘lsa (ro‘yxat, `range`, fayl satrlari). O‘qilishi oson.
- **while**: takrorlash sharti holatga bog‘liq bo‘lsa (foydalanuvchi kiritishi, sensor holati, tarmoqli javob kutilishi).

Tez-tez uchraydigan xatolar

1. **Off-by-one:** `range(1, n)` da `n` kirmasligini yodda tuting. Masalan, `1..n` uchun `range(1, n+1)`.
2. **Ro'yxatni aylanish chog'ida o'zgartirish:** element o'chirish/qo'shishda nusxa bilan yuring yoki yangi ro'yxat tuzing.
3. **while da o'zgaruvchini yangilamaslik:** cheksiz siklga olib keladi.
4. **Ketma-ketlik tartibi:** `set` tartib saqlamaydi — natija tartibi o'zgarishi mumkin.

Qo'shimcha: ixcham yozuv — comprehension

```
kvadratlar = [x*x for x in range(1, 6)]          # [1,4,9,16,25]
juftlar     = [x for x in range(10) if x % 2 == 0] # [0,2,4,6,8]
```

Xulosa

- **for** — ma'lum ketma-ketlik bo'ylab ixcham va xavfsiz yuradi.
- **while** — shartga asoslangan, takror soni noma'lum vaziyatlarda foydali.
- `range`, `enumerate`, `zip`, `dict.items()` amalda juda ko'p qo'l keladi.
- Off-by-one, ro'yxatni joyida o'zgartirish, `while` o'zgaruvchisini yangilamaslik — eng ko'p uchraydigan xatolar.

1-savol. 1 dan `n` gacha bo'lgan sonlarning yig'indisini hisoblang. `n` butun son ($n \geq 1$) sifatida kiritiladi. Ro'yxat va lug'atlardan hamda `sum()` funksiyasidan foydalanmang.

Kirish: bitta butun son `n`

Chiqish: Yigindi: <natija>

Namuna:

```
Kirish: 5
Chiqish: Yigindi: 15
```

Javob

```
n = int(input())
yigindi = 0
for i in range(1, n + 1):
    yigindi += i
print("Yigindi:", yigindi)
```

2-savol

1 dan `n` gacha bo'lgan **faqat juft sonlarni** ekranga chiqaring. `n` butun son ($n \geq 1$) sifatida kiritiladi. Ro'yxat va lug'atlardan foydalanmang.

Kirish: bitta butun son `n`

Chiqish: 1 dan `n` gacha bo'lgan juft sonlar, har biri alohida qatorda.

Namuna:

```
Kirish: 6
Chiqish:
2
4
6
```

Javob

```
n = int(input())
for i in range(1, n + 1):
    if i % 2 == 0:
        print(i)
```

3-savol

Foydalanuvchi ketma-ket sonlar kiritadi. Kiritish jarayoni **0** soni bilan tugaydi (0 hisobga olinmaydi). Siz kiritilgan barcha sonlarning **o'rtacha qiymatini** hisoblab chiqaring. Ro'yxat va lug'atlardan foydalanmang.

Kirish: ketma-ket musbat yoki manfiy butun sonlar, oxirida 0

Chiqish: O'rtacha: <natija>

Namuna:

```
Kirish:
5
15
10
0
Chiqish:
O'rtacha: 10.0
```

Izoh: $(5+15+10)/3 = 10.0$

Javob

```
jami = 0
sana = 0

while True:
    x = int(input())
    if x == 0:
        break
    jami += x
    sana += 1

if sana > 0:
    print("O'rtacha:", jami / sana)
else:
    print("Son kiritilmadi")
```

4-savol

Foydalanuvchi n sonini kiritadi. Siz 1 dan n gacha bo'lgan **toq sonlar yig'indisi** va **ularning soni** ni topib, **o'rtacha qiymatini** hisoblang. Ro'yxat va lug'atlardan foydalanmang.

Kirish: bitta butun son n ($n \geq 1$)

Chiqish: O'rtacha: <natija>

Namuna:

```
Kirish: 7
Chiqish:
O'rtacha: 4.0
```

Izoh: Toq sonlar = 1, 3, 5, 7 $\rightarrow$ yig'indi = 16, soni = 4, o'rtacha = $16/4 = 4.0$

Javob

```
n = int(input())
jami = 0
sana = 0

for i in range(1, n + 1):
    if i % 2 == 1:
        jami += i
        sana += 1

if sana > 0:
    print("O'rtacha:", jami / sana)
else:
    print("Toq son yo'q")
```

5-savol

Foydalanuvchi musbat butun son n kiritadi. Siz n sonini teskari tartibda yozib chiqishingiz kerak. Ro'yxat, lug'at va `str[::-1]` kabi qisqa usullardan foydalanmang. Faqat `while` sikli va arifmetik amallar (`%`, `//`) bilan ishleng.

Kirish: bitta butun son n ($n \geq 1$)

Chiqish: Teskari: <son>

Namuna:

```
Kirish: 12340
Chiqish:
Teskari: 4321
```

Izoh: 12340 sonining raqamlari teskari tartibda yozilganda 4321 hosil bo'ladi (boshlang'ich nol tashlanadi).

Javob

```
n = int(input())
teskari = 0

while n > 0:
    qoldiq = n % 10
    teskari = teskari * 10 + qoldiq
    n //= 10

print("Teskari:", teskari)
```

6-savol

Foydalanuvchi musbat butun son n kiritadi. Siz 1 dan n gacha bo'lgan barcha **tub sonlarni** (faqat 1 va o'ziga bo'linadigan sonlar) ekranga chiqarishingiz kerak. Ro'yxat va lug'atlardan foydalanmang, faqat `for` va `while` sikllaridan foydalaning.

Kirish: bitta butun son n ($n \geq 2$)

Chiqish: 1 dan n gacha bo'lgan tub sonlar, har biri alohida qatorda.

Namuna:

```
Kirish: 10
Chiqish:
2
3
5
7
```

Izoh: 1 tub son emas, shuning uchun chiqmaydi.

Javob

```
n = int(input())

for i in range(2, n + 1):      # 2 dan n gacha bo'lgan sonlarni tekshiramiz
    tub = True
    j = 2
    while j * j <= i:          # faqat ildizigacha tekshirish kifoya
        if i % j == 0:
            tub = False
            break
        j += 1
    if tub:
        print(i)
```

7-savol

Foydalanuvchi musbat butun son n kiritadi. Siz 1 dan n gacha bo'lgan barcha **tub sonlarni Eratosfen g'alviri** (Sieve of Eratosthenes) usuli yordamida topib, ekranga chiqaring.

Kirish: bitta butun son n ($n \geq 2$)

Chiqish: 1 dan n gacha bo'lgan tub sonlar, har biri alohida qatorda.

Namuna:

```
Kirish: 20
Chiqish:
2
3
5
7
11
13
17
19
```

Javob

```
n = int(input())

# Barcha sonlarni dastlab tub deb faraz qilamiz (True)
tub = [True] * (n + 1)
tub[0] = tub[1] = False # 0 va 1 tub emas

p = 2
while p * p <= n:
    if tub[p]:
        # p ning barcha ko'paytmalari tub emas
        i = p * p
        while i <= n:
            tub[i] = False
            i += p
        p += 1

# Natijani chiqarish
for i in range(2, n + 1):
    if tub[i]:
        print(i)
```

2. Break va Continue operatorlarining qo'llanilishi

1. break operatori

`break` operatori sikl (`for`, `while`) ichida ishlatiladi va siklni **darhol to'xtatadi**.

- Ko'pincha izlanayotgan qiymat topilganda yoki shart bajarilganda siklni ortiqcha ishlatmaslik uchun qo'llaniladi.

Misol:

```
for i in range(1, 10):  
    if i == 5:  
        break  
    print(i)
```

Natija:

```
1  
2  
3  
4
```

Bu yerda `i == 5` bo'lganda sikl to'xtab qoldi.

2. continue operatori

`continue` operatori esa siklni to'xtatmaydi, balki **joriy iteratsiyani tashlab**, keyingi iteratsiyaga o'tadi.

- Ko'pincha kerakmas qiymatlarni tashlab ketishda ishlatiladi.

Misol:

```
for i in range(1, 10):  
    if i % 2 == 0:  
        continue  
    print(i)
```

Natija:

```
1  
3  
5  
7  
9
```

Juft sonlar tashlab o'tildi, faqat toq sonlar chiqdi.

3. break va continuening farqi

Operator	Vazifasi
----------	----------

break	siklni butunlay to'xtatadi
continue	Faqat joriy iteratsiyani tashlab o'tadi, sikl davom etadi

1-savol (break)

Savol: Quyidagi kodni bajarganda qanday natija chiqadi va nechta iteratsiya bajariladi?

```
names = ["Ali", "Vali", "STOP", "Hasan", "Husan"]

for i, name in enumerate(names):
    if name == "STOP":
        print("To'xtash nuqtasi topildi:", i)
        break
    print("Qabul qilindi:", name)
```

Javob:

```
Qabul qilindi: Ali
Qabul qilindi: Vali
To'xtash nuqtasi topildi: 2
```

Jami **3 ta iteratsiya** bajarildi ($i = 0, 1, 2$).

2-savol (continue)

Savol: Quyidagi kodni bajarganda konsolda nima chiqadi va qaysi qiymatlar tashlab o'tiladi?

```
for n in range(1, 11):
    if n % 3 == 0:
        continue
    print("n:", n, "- kvadrati:", n*n)
```

Javob:

```
n: 1 - kvadrati: 1
n: 2 - kvadrati: 4
n: 4 - kvadrati: 16
n: 5 - kvadrati: 25
n: 7 - kvadrati: 49
n: 8 - kvadrati: 64
n: 10 - kvadrati: 100
```

3, 6, 9 qiymatlari tashlab o'tildi, chunki ular 3 ga bo'linadi.

3-savol: Foydalanuvchi kiritgan sonlar ichidan **birinchi manfiy sonni** topuvchi dastur yozing. Dastur manfiy son topilganda siklni darhol to'xtatishi kerak.

Javob (kod):

```
numbers = [10, 5, 3, -2, 7, -5]

for n in numbers:
```

```
if n < 0:
    print("Birinchi manfiy son:", n)
    break
```

4-savol: 1 dan 20 gacha bo'lgan sonlar ichidan **faqat 3 ga karrali sonlarning kvadratini chiqaruvchi** dastur yozing. Boshqa sonlar tashlab o'tilishi kerak.

Javob (kod):

```
for i in range(1, 21):
    if i % 3 != 0:
        continue
    print(i, "ning kvadrati:", i*i)
```

5-savol: Foydalanuvchi kiritgan son tub sonmi yoki yo'qligini aniqlovchi dastur yozing. Agar son tub bo'lmasa, dastur darhol `break` orqali tekshiruvni to'xtatishi kerak.

Javob (kod):

```
n = 29 # foydalanuvchi kiritgan son
for i in range(2, n):
    if n % i == 0:
        print("Tub emas, bo'linuvchi:", i)
        break
else:
    print("Tub son")
```

6-savol: 1 dan 50 gacha bo'lgan sonlar orasidan faqat **juft sonlarni chiqarish**, lekin ular orasida **5 ga karrali sonlarni tashlab o'tuvchi** dastur yozing.

Natijada 2, 4, 6, 8, ... kabi juft sonlar chiqadi, lekin **10, 20, 30, 40, 50** chiqarilmaydi.

Javob (kod):

```
for i in range(1, 51):
    if i % 2 != 0:
        continue # faqat juftlarni qoldiramiz
    if i % 5 == 0:
        continue # 5 ga karralilarni tashlab o'tamiz
    print(i)
```


